

On Neural Generative Models

Edinburgh Summer School on Generative AI for Extremes

Miguel de Carvalho

University of Edinburgh

8–11 September 2026

JCMB, Edinburgh



THE UNIVERSITY
of EDINBURGH



Scope

Positioning the Lecture

- This lecture provides an introduction to [neural generative modeling](#), viewed primarily through a statistical lens.
- As noted earlier, we do *not* consider extremes just yet.
- The goal is to introduce some mainstream tools underlying modern generative models.
- Given the natural interface between

SIMULATION

GENERATION

we will start with some introductory [probabilistic](#), [statistical](#), and [algorithmic](#) notions associated with these.

Agenda

Setting the Stage

- Introduction: Background On Simulation-Based Notions

Agenda

Setting the Stage

- Introduction: Background On Simulation-Based Notions

Also Covered in This Lecture

- Normalizing Flows
- Transformers
- Autoencoders and VAEs
- Challenges with Extremes

Agenda

Setting the Stage

- Introduction: Background On Simulation-Based Notions

Also Covered in This Lecture

- Normalizing Flows
- Transformers
- Autoencoders and VAEs
- Challenges with Extremes

Covered in Next Lectures

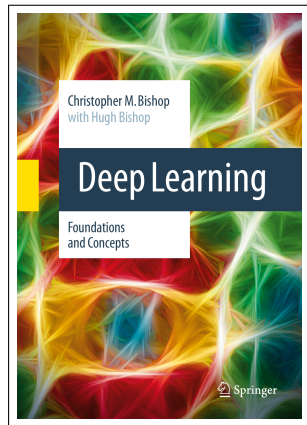
- Diffusion Models (Sabanis)
- GANs (Generative Adversarial Networks) (Girard; Trapin; Richards)

Sources for These Notes

Background Reading

- These slides draw on [personal notes](#).
- They also draw on [Bishop and Bishop \(2024\)](#), especially Chapters 12, 18 and 19.

Christopher M. Bishop and Hugh Bishop
Deep Learning: Foundations and Concepts
Springer, 2024.



Introduction: Simulation, Generation, and Modeling

What Should a Generative Model Achieve?

In many cases (but not all—more on this below), generated samples $\mathbf{X}^* \sim F_{\hat{\theta}}$, where the parameter $\hat{\theta}$ is learned from training data $X_1, \dots, X_n \stackrel{\text{iid}}{\sim} F$, should exhibit three complementary properties.

- **Fidelity**: Generated samples resemble observations from the underlying data distribution.
- **Diversity**: Generated samples have a reasonable degree of variability, and do not collapse to a limited set of outcomes.
- **Novelty**: Generated samples are not merely copies of training observations, but new and plausible outcomes.

Example (Synthetic Storm Fields)

A few realistic storms generated repeatedly have fidelity but little diversity; pixel-for-pixel training replicas have little novelty.

Introduction: Simulation, Generation, and Modeling

The Bootstrap as an Empirical Generator

Given $X_1, \dots, X_n \stackrel{\text{iid}}{\sim} F$, estimate F by the **empirical distribution function**

$$\hat{F}_n(x) = \frac{1}{n} \sum_{i=1}^n I(X_i \leq x).$$

Conditional on the observations, either construction generates from $\hat{F}_n$:

$$U \sim \text{Unif}(0, 1), \quad X^* = \hat{F}_n^{-1}(U), \quad \text{or} \quad J \sim \text{Unif}\{1, \dots, n\}, \quad X^* = X_J.$$

Repeating this independently n times gives a **bootstrap sample**, $\mathbf{X}^*$.

- **Fidelity**: Conditional on the data, the generated value has distribution function $\hat{F}_n$. The **Glivenko–Cantelli theorem** gives $\sup_x |\hat{F}_n(x) - F(x)| \rightarrow 0$ almost surely.
- **Diversity**: Resamples vary, but each generated value is one of $X_1, \dots, X_n$.
- **Novelty**: Multiplicities and ordering can change, but no generated value is new.

Introduction: Simulation, Generation, and Modeling

Generating Data from a Marginal Distribution—The Inversion Method

- Let F be a distribution function and $U \sim \text{Unif}(0, 1)$. Then

$$X^* = F^{-1}(U) \sim F,$$

where $F^{-1}(p) = \inf\{x : F(x) \geq p\}$ denotes quantile function.

- Hence, if $U_1, \dots, U_m \stackrel{\text{iid}}{\sim} \text{Unif}(0, 1)$, then

$$X_1^* = F^{-1}(U_1), \dots, X_m^* = F^{-1}(U_m) \stackrel{\text{iid}}{\sim} F.$$

This is a **random sample** from F .

- In many applications, however, F is typically unknown. If $F = F_\theta$, with $\theta \in \Theta \subseteq \mathbb{R}^q$, we estimate θ from the training data and simulate instead from

$$X^* = F_{\hat{\theta}}^{-1}(U) \sim F_{\hat{\theta}}.$$

- This **plug-in generator** conditions on $\hat{\theta}$ and does not propagate estimation uncertainty in θ .

Introduction: Simulation, Generation, and Modeling

Independent Generation

- Independent draws from a marginal distribution—as those yielded by the inversion method—can be represented graphically as shown below.
- In the [graph](#) below, [nodes](#) represent random variables whose values we simulate, while [arrows](#) (absent here) indicate direct dependencies between them.



- Similar graphs will appear below to visually represent the dependence structure underlying other models.

Introduction: Simulation, Generation, and Modeling

From Marginal to Conditional Simulation

- The **inversion method** provides a simple way to generate independent draws from a target marginal distribution.
- In many applications, however, the distribution of the next draw depends on what has already been observed or generated.
- In an autoregressive language model, for example, the distribution of the next token is described by the conditional distribution function

$$F_n(x \mid X_1^*, \dots, X_{n-1}^*),$$

which depends on the preceding context.

- This raises the question:

How do we simulate from a conditional distribution?

Regression provides a natural first example. In particular, **linear regression** can be represented as a **neural network** with a single linear layer (e.g., Bishop and Bishop, 2024), making it a useful bridge to neural generative models.

Introduction: Simulation, Generation, and Modeling

A Regression Model as a Generator, I

Example (Gaussian Linear Regression as a Generator, I)

Let $X \sim \text{Unif}(0, 1)$ and

$$Y = \alpha + \beta X + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2), \quad \varepsilon \perp X,$$

where α, β and $\sigma > 0$ are known. Thus $Y \mid X = x \sim N(\alpha + \beta x, \sigma^2)$. Generate a synthetic data set by drawing, independently for $i = 1, \dots, n$,

$$X_i^* \sim \text{Unif}(0, 1), \quad \varepsilon_i^* \sim N(0, \sigma^2), \quad Y_i^* = \alpha + \beta X_i^* + \varepsilon_i^*.$$

The pairs

$$\{(X_i^*, Y_i^*)\}_{i=1}^n$$

are iid from the joint distribution with distribution function $F_{X,Y}$.

For [conditional simulation](#) at a fixed x , draw only $Y^* = \alpha + \beta x + \varepsilon^*$.

Introduction: Simulation, Generation, and Modeling

A Regression Model as a Generator, II

Consider the same setup as in the previous example, but suppose $\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n$ is observed. Under the same Gaussian model, the **OLS (ordinary least squares)** estimator is

$$(\hat{\alpha}, \hat{\beta}) = \arg \min_{a, b} \sum_{i=1}^n (y_i - a - bx_i)^2, \quad \hat{\sigma}^2 = \frac{\sum_{i=1}^n (y_i - \hat{\alpha} - \hat{\beta}x_i)^2}{n-2}.$$

The fitted conditional generator at a specified value x is

$$Y^* = \hat{\alpha} + \hat{\beta}x + \varepsilon^*, \quad \varepsilon^* \sim N(0, \hat{\sigma}^2).$$

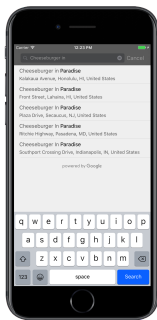
Example (Stochastic Regression Imputation (SRI))

If $X_i = x_i$ is observed and Y_i is missing, use this draw to impute Y_i . Fresh noise retains **conditional response variability**. Using only $\hat{\alpha} + \hat{\beta}x_i$ removes that variability.

Introduction: Simulation, Generation, and Modeling

Generating One Element at a Time

An important aspect of generative modelling is the ability to generate data sequentially—as in autocomplete on a mobile phone or text generation by a large language model.



Autocomplete



Next-token generation

At each step, generation is governed by a distribution for the next element given what has already been generated. This leads naturally to the **notion of predictive distribution**.

Introduction: Simulation, Generation, and Modeling

The Distribution of What Comes Next

After observing $x_1, \dots, x_n$, uncertainty about the next value is described by the **one-step-ahead predictive distribution function**

$$F_{n+1}(x \mid x_1, \dots, x_n) = \Pr(X_{n+1} \leq x \mid X_1 = x_1, \dots, X_n = x_n).$$

Conditional inversion then gives a simple sequential generator:

$$U_{n+1} \sim \text{Unif}(0, 1), \quad X_{n+1}^* = F_{n+1}^{-1}(U_{n+1} \mid X_1^*, \dots, X_n^*).$$

The generated value is appended to the context and the procedure is repeated:

$$\underbrace{(X_1^*, \dots, X_n^*)}_{\text{current context}} \xrightarrow{\text{append } X_{n+1}^*} \underbrace{(X_1^*, \dots, X_n^*, X_{n+1}^*)}_{\text{updated context}}.$$

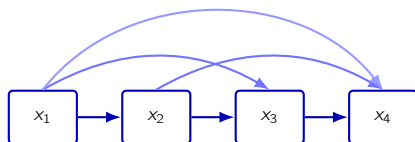
For text generation, it assigns probabilities to the possible next tokens—and, as will be seen later in this workshop, the same conditional framework is natural for cascading extremes.

Introduction: Simulation, Generation, and Modeling

Autoregressive Factorisation

Repeated application of the product rule gives the autoregressive factorisation

$$f(x_1, \dots, x_n) = f(x_1) \prod_{i=2}^n f(x_i \mid x_1, \dots, x_{i-1}).$$



For this ordering, the dense graph represents the chain rule. Omitting an incoming edge removes that variable from the relevant conditional density and imposes a conditional-independence restriction.

Independence

First order

$$f(x_1, \dots, x_n) = \prod_{i=1}^n f(x_i).$$

$$f(x_1, \dots, x_n) = f(x_1) \prod_{i=2}^n f(x_i \mid x_{i-1}).$$

Introduction: Simulation, Generation, and Modeling

A Generalized Linear Model Is the Starting Point

A generalized linear model maps the input directly to a conditional response distribution.

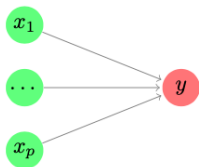
Input

Output

For a response family with conditional mean $\mu_\beta(x)$ and dispersion ϕ ,

$$F_{Y|X=x}(y) = F\{y; \mu_\beta(x), \phi\},$$

$$g\{\mu_\beta(x)\} = \beta_0 + x^\top \beta.$$



Direct input–output map

- Gaussian response with the identity link gives linear regression.
- Bernoulli–logit and Poisson–log models keep the same direct architecture.

Once fitted, the model can also be used as a conditional generator, subject to the earlier disclaimer:

$$U \sim \text{Unif}(0, 1), \quad Y^* = F^{-1}\{U; \mu_{\hat{\beta}}(x), \hat{\phi}\}.$$

Introduction: Simulation, Generation, and Modeling

The Same Structure, Different Language

The GLM can be read almost verbatim in neural-network language:

Statistics / GLM

ML / neural network

Covariate x_j

Feature x_j

Regression coefficient β_j

Weight w_j

Intercept β_0

Bias b

Linear predictor η

Pre-activation a

Inverse link g^{-1}

Output activation ρ

Inverse multinomial logit

Softmax

For a single output unit,

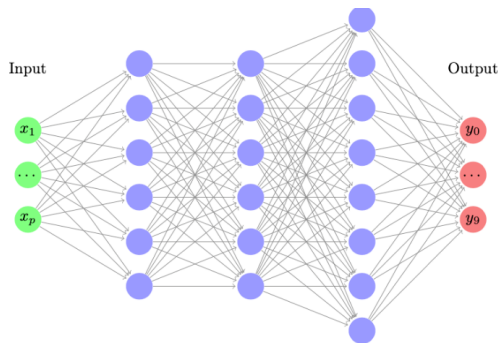
$$a = \beta_0 + x^\top \beta, \quad \mu(x) = g^{-1}(a),$$

so g^{-1} acts as the output activation.

A hidden layer then allows the model to have further flexibility.

Introduction: Simulation, Generation, and Modeling

A Hidden Layer Adds Further Flexibility



A deep network with three hidden layers

The conditional model remains

$$F_{Y|X=x}(y) = F\{y; \mu_{\theta}(x), \phi\}.$$

Removing the hidden layers recovers the GLM; adding them gives the model further flexibility to fit a broader class of data patterns.

Each hidden layer forms a weighted sum and applies a nonlinear activation:

$$h^{(0)}(x) = x,$$

$$a^{(\ell)}(x) = W_{\ell}h^{(\ell-1)}(x) + b_{\ell},$$

$$h^{(\ell)}(x) = \rho_{\ell}\{a^{(\ell)}(x)\},$$

for $\ell = 1, \dots, L$. The output layer supplies the predictor:

$$g\{\mu_{\theta}(x)\} = \alpha_0 + \alpha^{\top} h^{(L)}(x).$$

Normalizing Flows

From Inversion to a Learned Transformation

- **Scalar inversion:** Let F be strictly increasing on its support interval I , with continuous density $f > 0$. Then $F^{-1} : (0, 1) \rightarrow I$ is continuously differentiable, and

$$U \sim \text{Unif}(0, 1) \implies X^* = F^{-1}(U) \sim F.$$

- **Multivariate transport:** For $D > 1$, there is no unique analogue of the scalar quantile function. Choose a simple base law F_U —often $N(\mathbf{0}, I_D)$, although a uniform law is also possible—and set

$$\mathbf{U} \sim F_U \implies \mathbf{X}^* = T_\theta(\mathbf{U}) \sim F_\theta.$$

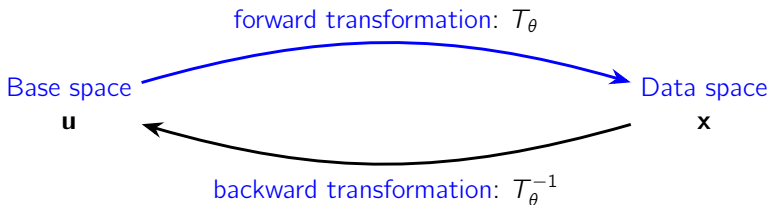
- **Normalizing flow:** Learn T_θ from data within a family of smooth bijections satisfying $\det J_{T_\theta}(\mathbf{u}) \neq 0$.

Thus a flow learns a multivariate version of this construction.

Normalizing Flows

Invertibility Gives the Density

The same transformation can be read in two directions:



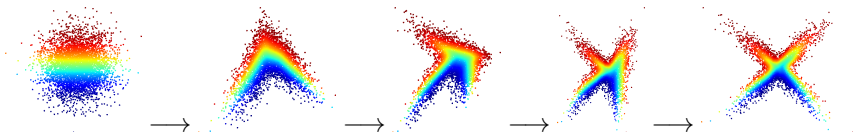
If F_U has density f_U and $\mathbf{X} = T_\theta(\mathbf{U})$, then the change-of-variables formula gives

$$f_\theta(\mathbf{x}) = f_U(\mathbf{u}) |\det J_{T_\theta}(\mathbf{u})|^{-1}, \quad \mathbf{u} = T_\theta^{-1}(\mathbf{x}).$$

For a uniform base, $f_U(\mathbf{u}) = 1$ inside the unit cube. More generally, f_U gives the base density, while the Jacobian determinant corrects for the local expansion or contraction of volume.

Normalizing Flows

Simple Maps Become Flexible through Composition



Starting with $\mathbf{U}_0 \sim F_U$, let

$$\mathbf{U}_k = T_k(\mathbf{U}_{k-1}; \theta_k), \quad k = 1, \dots, K, \quad \mathbf{X} = \mathbf{U}_K.$$

Then $\theta = (\theta_1, \dots, \theta_K)$, $T_\theta = T_K \circ \dots \circ T_1$, and

$$\log f_\theta(\mathbf{x}) = \log f_U(\mathbf{u}_0) - \sum_{k=1}^K \log |\det J_{T_k}(\mathbf{u}_{k-1}; \theta_k)|.$$

Each $T_k(\cdot; \theta_k)$ is typically parameterised using a neural network within an invertible architecture, chosen so that its inverse and Jacobian determinant are tractable. T_θ is the **forward transformation** from base to data space; T_θ^{-1} is the **backward transformation** that normalises data.

Normalizing Flows

Fit Backwards, Generate Forwards

For observations $\mathbf{X}_1, \dots, \mathbf{X}_n$, write $\mathbf{u}_i = T_{\theta}^{-1}(\mathbf{X}_i)$. The explicit density permits maximum-likelihood estimation:

$$\hat{\theta} = \arg \max_{\theta} \sum_{i=1}^n [\log f_U(\mathbf{u}_i) - \log |\det J_{T_{\theta}}(\mathbf{u}_i)|].$$

Backward transformation

Fitting

$$\mathbf{X}_i \xrightarrow{T_{\theta}^{-1}} \mathbf{u}_i$$

Map observations to the base space
and maximise the likelihood.

Forward transformation

Generation

$$\mathbf{U}^* \sim F_U \xrightarrow{T_{\hat{\theta}}} \mathbf{X}^*$$

Draw from the base distribution and
run the fitted flow forwards.

As before, generation means drawing from a simple law and transforming.

Normalizing Flows

Flexible Tails for Normalizing Flows

We have not yet specialised to extremes. Hickling and Prangle show how tail behaviour can enter the flow itself.

Flexible Tails for Normalizing Flows

Samuël Hickling^{1,2} Denis Prangle^{1,2}

Abstract

Normalizing flows are a flexible class of probability distributions, expressed as transformation of a simple base distribution. A limitation of standard normalizing flows is representing distributions with heavy tails, which arise in applications to both density estimation and variational inference. A popular current solution to this problem is to use a heavy-tailed base distribution. We argue this can lead to poor performance due to the difficulty of optimizing neural networks. We propose an alternative, 'tail transform flow' (TF), which uses a Gaussian base distribution and a final transformation layer which can produce heavy tails. Experiments of results show this approach outperforms current methods, especially when the target distribution has large dimensions or fat right-tails.

1. Introduction

A normalizing flow (NF) sequence is a complex probability distribution as a parametric transformation of a simple base distribution. A NF sample is

$$x = T(x_0), \quad (1)$$

where x_0 is a sample from the base distribution, typically $N(0, I)$. A number of transformations have been proposed which produce flexible multivariate distributions. Typically flexible transformations are composed to form T with the desired level of flexibility. Approaches include: density estimation (fitting a transformation to observed data points) and variational inference (fitting a transformation to a target distribution). In either case θ , the parameters controlling T , can be optimized using stochastic gradient methods for a suitable objective function. For reviews of NFs see Roberts et al. (2023) and Papapanagiotou et al. (2023).

1.1. Tail of Multivariate Distributions. University of Bristol, Bristol, UK. Correspondence to: Samuël Hickling (s.hickling@bristol.ac.uk).

Proceedings of the 41st International Conference on Machine Learning, Vancouver, Canada, PMLR 202, 2023. Copyright 2023 by the authors.

Hickling and Prangle (2025)

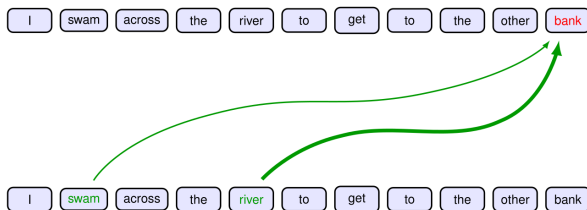
Limitation. A Gaussian base followed by standard Lipschitz layers remains light-tailed.

Proposal. Tail transform flow retains the Gaussian base and appends an EVT-motivated non-Lipschitz layer, with separate upper- and lower-tail parameters. The main flow models dependence; the final layer controls marginal tails.

Evidence and open problems: In the experiments reported by Hickling and Prangle, gains over the chosen baselines were largest in higher dimensions and for heavier-tailed targets. Extremal dependence remains a separate question.

Transformers

Context Changes the Meaning of a Token



Example (The Word *Bank*)

Before seeing a sentence, *bank* has one learned input embedding. Here, *swam* and *river* give it the meaning of riverbank; in “deposit money at the bank”, different words indicate a financial institution.

Self-attention updates each token representation by combining information from the available context. The contribution of each position is recomputed for every input sequence.

Transformers

Attention as an Input-Dependent Weighted Average

Let $X \in \mathbb{R}^{N \times D}$ have row n equal to x_n^T . For each output position n , attention forms

$$y_n = \sum_{m=1}^N a_{n,m}(X) x_m, \quad a_{n,m}(X) \geq 0, \quad \sum_{m=1}^N a_{n,m}(X) = 1.$$

- **Weighted average:** For fixed n , y_n is a convex combination of the input token vectors.
- **Position-specific:** Each output position has its own N attention coefficients.
- **Input-dependent:** The coefficients change with X , so the combination changes from one sequence to another.

Self-attention determines these coefficients by comparing position n with every position m .

Transformers

Self-Attention in Three Steps

A trainable attention head maps the tokens to **queries**, **keys** and **values**:

$$q_n = W^{(q)\top} x_n \in \mathbb{R}^{D_k}, \quad k_m = W^{(k)\top} x_m \in \mathbb{R}^{D_k}, \quad v_m = W^{(v)\top} x_m \in \mathbb{R}^{D_v}.$$

- **Compare**: score how relevant position m is to position n ,
 $s_{n,m} = q_n^\top k_m / \sqrt{D_k}.$

- **Normalise**: turn the scores into non-negative weights,

$$a_{n,m} = \frac{\exp(s_{n,m})}{\sum_{\ell=1}^N \exp(s_{n,\ell})}, \quad \sum_{m=1}^N a_{n,m} = 1.$$

- **Combine**: update the token by a weighted average, $y_n = \sum_{m=1}^N a_{n,m} v_m.$

Example (Film Retrieval)

A query describes a viewer's preferences, each key describes a film, and the corresponding value is its content. Better query-key matches receive larger weights.

Transformers

Position Must Be Added to Attention

Unmasked self-attention without position information is **permutation equivariant**:

$$\mathcal{T}(PX) = P\mathcal{T}(X)$$

for any permutation matrix P . Reordering the input only reorders the output.

Consequently, under a permutation-invariant readout, token permutations such as the following are indistinguishable:

“The food was **bad**, not good at all”

from

“The food was **good**, not bad at all.”

Position is introduced by replacing each token embedding with, for example,

$$\tilde{\mathbf{x}}_n = \mathbf{x}_n + \mathbf{r}_n,$$

where $\mathbf{r}_n$ is a fixed or learned **positional encoding** (Bishop and Bishop, 2024, Chapter 12).

Attention learns which tokens interact; positional encoding tells it where they are.

Transformers

Encoders and Decoders Use Attention Differently

	Encoder	Decoder
Visibility	All positions	Prefix only (causal mask)
Block output	Contextual states	Causal contextual states
Typical role	Representation or prediction	Sequential generation

In causal self-attention, the attention weight from position n to position m is set to zero whenever $m > n$. An output head maps the resulting causal state to next-token logits.

Transformers

A Decoder Returns a Next-Token Distribution

Given a prefix $x_1, \dots, x_n$, the decoder produces logits $\mathbf{o}_n = (o_{n,1}, \dots, o_{n,K})^\top$ and hence

$$\begin{aligned}\pi_{n,k} &:= \Pr_{\hat{\theta}}\{X_{n+1} = k \mid X_1 = x_1, \dots, X_n = x_n\} \\ &= [\text{softmax}(\mathbf{o}_n)]_k, \quad k = 1, \dots, K.\end{aligned}$$

prefix $x_1, \dots, x_n \longrightarrow$ decoder $\longrightarrow$ logits $\mathbf{o}_n \longrightarrow$ softmax $\longrightarrow \boldsymbol{\pi}_n$

The causal mask allows all conditional distributions used for training to be evaluated in parallel, without revealing future tokens.

Generation is nevertheless sequential:

$$X_{n+1}^* \sim \text{Categorical}(\boldsymbol{\pi}_n),$$

after which the generated value is appended to the context and the procedure is repeated.

The decoder supplies a predictive distribution; a sampling rule turns it into the next token.

Transformers

Sampling from the Predictive Distribution

Rule	Consequence
Direct sampling	Preserves the fitted conditional distribution.
Greedy search	Deterministic; selects the largest probability.
Top- K or top- p	Truncates and renormalises the distribution.
Temperature	Concentrates or flattens before sampling.

For temperature $T > 0$,

$$\pi_{n,k}^{(T)} = \frac{\exp(o_{n,k}/T)}{\sum_{j=1}^K \exp(o_{n,j}/T)}.$$

Here $T = 1$ leaves the fitted law unchanged. Unless all logits are equal, $T < 1$ lowers its entropy, whereas $T > 1$ moves it towards the uniform distribution and raises its entropy.

The sampling rule is part of the generator: it determines how the predictive distribution becomes a sequence.

Variational Autoencoders

Generation Begins with Two Draws

To generate a sample from a random vector X , a VAE uses a **two-step sampling strategy**:

- **Latent draw**: sample $Z^* \sim p_\alpha$ from a prior distribution, whose parameter α may be fixed or learned.
- **Conditional draw**: given Z^* , sample $X^* \sim p_\theta(\cdot | Z^*)$.

A VAE specifies a parametric conditional model $p_\theta(x | z)$, called the **likelihood** or **probabilistic decoder**; θ contains learnable parameters. Together, the two stages define

$$p_{\alpha,\theta}(x) = \int p_\theta(x | z) p_\alpha(z) dz.$$

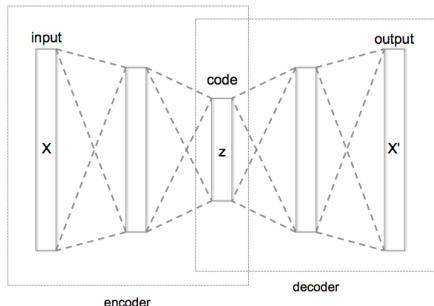
Key Point

The prior p_α and decoder p_θ define the marginal model $p_{\alpha,\theta}(x)$. The variational distribution $q_\phi(z | x)$ approximates its posterior and yields an optimisable lower bound.

Autoencoders

The Input Is Also the Target

An autoencoder is a neural model trained to **predict, or reconstruct, its own input**.



For $x \in \mathbb{R}^D$ and $z \in \mathbb{R}^M$,

$$z = e_{\phi}(x), \quad \tilde{x} = d_{\theta}(z),$$

and a common objective is

$$\min_{\theta, \phi} \frac{1}{n} \sum_{i=1}^n \|x_i - d_{\theta}\{e_{\phi}(x_i)\}\|^2.$$

An undercomplete code has $M < D$, so the representation is lower-dimensional; regularisation may still be needed to prevent memorisation.

Boehmke and Greenwell, *Hands-On Machine Learning with R*, Figure 19.1.

Variational Autoencoders

The Variational Distribution Approximates the Posterior

The generative model runs from z to x . Once x is observed, inference about its latent code requires the posterior distribution

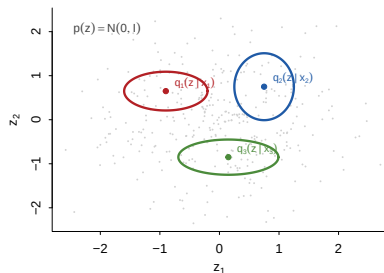
$$p(z | x) = \frac{p_{\theta}(x | z)p_{\alpha}(z)}{\int p_{\theta}(x | u)p_{\alpha}(u) du}.$$

The denominator is generally intractable. A VAE therefore introduces the **variational distribution**

$$q_{\phi}(z | x) \approx p(z | x).$$

The distribution $q_{\phi}(z | x)$, also called the probabilistic encoder, approximates the posterior for every observation.

Training optimises θ and ϕ , and also α when the prior parameter is learned, by maximising the ELBO.



Variational Autoencoders

The Evidence Lower Bound Trains the Model

Given observations $x^{(1)}, \dots, x^{(n)}$, for every i ,

$$\log p_{\alpha, \theta}\{x^{(i)}\} \geq \mathcal{L}\{x^{(i)}, \alpha, \theta, \phi\},$$

where $\mathcal{L}$ denotes the **evidence lower bound** (ELBO):

$$\mathcal{L}\{x^{(i)}, \alpha, \theta, \phi\} = \underbrace{-D_{\text{KL}}\{q_{\phi}(z | x^{(i)}) \parallel p_{\alpha}(z)\}}_{\text{Kullback-Leibler penalty}} + \underbrace{\mathbb{E}_{q_{\phi}(z|x^{(i)})}[\log p_{\theta}\{x^{(i)} | Z\}]}_{\text{expected log likelihood}}.$$

Here

$$D_{\text{KL}}(q \parallel p) = \int q(z) \log \frac{q(z)}{p(z)} dz \geq 0.$$

- The first term penalises the divergence of $q_{\phi}(z | x)$ from the prior $p_{\alpha}(z)$ used for generation.
- The second rewards high conditional log likelihood under $p_{\theta}(x | z)$.

Training maximises $n^{-1} \sum_{i=1}^n \mathcal{L}\{x^{(i)}, \alpha, \theta, \phi\}$.

Variational Autoencoders

A Standard Gaussian Parameterisation

Example (Standard VAE)

For $z \in \mathbb{R}^M$ and $x \in \mathbb{R}^D$, the common parameterisation is

$$\begin{aligned}p(z) &= \mathcal{N}(z; 0, I_M), \\p_\theta(x | z) &= \mathcal{N}\{x; \mu_\theta(z), \text{diag}\{\sigma_\theta(z)\}^2\}, \\q_\phi(z | x) &= \mathcal{N}\{z; \mu_\phi(x), \text{diag}\{\sigma_\phi(x)\}^2\}.\end{aligned}$$

The functions μ_θ , σ_θ , μ_ϕ and σ_ϕ are neural networks.

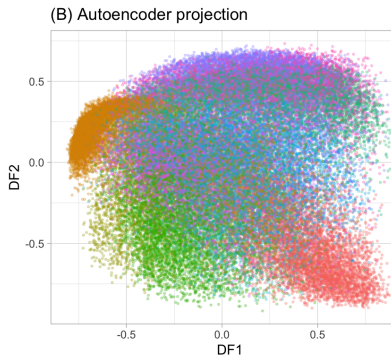
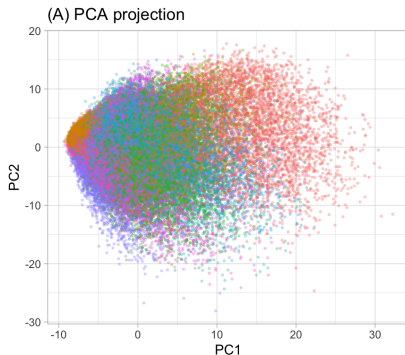
To differentiate through a draw from $q_\phi(z | x)$, write

$$\varepsilon \sim \mathcal{N}(0, I_M), \quad Z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon.$$

Variational Autoencoders

A Dimension-Reduction Perspective

- Another way to think of autoencoders is as a **dimension-reduction approach**. From this point of view, they have close links with **principal component analysis (PCA)**.



Challenges with Extremes

Generative Weather Models Are Developing Rapidly



Challenges with Extremes

A Special Issue on EVT and AI



- *Bridging heavy tails and artificial intelligence*, *Extremes*, 29(2), June 2026, edited by Miguel de Carvalho and Jordan Richards.
- The nine research articles include three contributions on generative modelling of threshold exceedances or angular components.

Challenges with Extremes

Angular Simulation with Neural Generators

Extremes
https://doi.org/10.1007/s10687-025-00522-7

RESEARCH

Generative machine learning for multivariate angular simulation

Jakob Benjamin Wessel¹ · Callum J. R. Murphy-Barltrop^{2,3} · Emma S. Simpson⁴

Received: 20 April 2025 / Revised: 21 November 2025 / Accepted: 4 December 2025
© The Author(s) 2025

Abstract

With the recent development of new geometric and angular-radial frameworks for multivariate extremes, reliably simulating from angular variables in moderate-to-high dimensions is of increasing importance. Empirical approaches have the benefit of simplicity, and work reasonably well in low dimensions, but as the number of variables increases, they can lack the required flexibility and scalability. Classical parametric models for angular variables, such as the von Mises–Fisher distribution (vMF), provide an alternative. Exploiting finite mixtures of vMF distributions increases their flexibility, but there are cases where, without letting the number of mixture components grow considerably, a mixture model with a fixed number of components is not sufficient to capture the intricate features that can arise in data. Owing to their flexibility, generative deep learning methods are able to capture complex data structures; they therefore have the potential to be useful in the simulation of multivariate angular variables. In this paper, we introduce a range of deep learning approaches for this task, including generative adversarial networks, normalizing flows and flow matching. We assess their performance via a range of metrics, and make comparisons to the more classical approach of using a finite mixture of vMF distributions. The methods are also applied to a meteorological data set, with diagnostics indicating strong performance, demonstrating the applicability of such techniques to real-world, complex data structures.

Keywords Angular simulation · Deep generative models · Multivariate extremes · Neural networks

✉ Jakob Benjamin Wessel
j.wessel@exeter.ac.uk

¹ Department of Mathematics and Statistics, University of Exeter, Exeter, UK

² Institut für Mathematische Stochastik, Technische Universität Dresden, Dresden, Germany

³ Center for Scalable Data Analytics and Artificial Intelligence (ScaDS.AI), Dresden/Leipzig, Germany

⁴ Department of Statistical Science, University College London, London, UK

Published online: 29 December 2025



- **Task:** Learn and simulate the angular component in radial–angular models for multivariate extremes.
- **Comparison:** GANs, normalizing flows and flow matching are compared with finite mixtures of von Mises–Fisher distributions.
- **Division of labour:** The neural generator learns directions of joint extremes; an EVT radial model supplies tail extrapolation.

Featured on Thursday: *Multivariate Angular Simulation* (Emma Simpson).

Also on Thursday: *Transformers for extremes in Cascades, Multivariate Extremes, and Transformers.*

(Johnny Lee; Clemente Ferrer)

Wessel, Murphy-Barltrop and
Simpson (2026)

Closing Remarks

One Simulation Principle across Architectures

- These notes presented an overview of the statistical ideas underlying several neural generative models.
- **Normalizing flows** transport a simple base draw through an invertible learned map.
- **Decoder transformers** parameterise next-token distributions; a sampling or decoding rule extends the sequence.
- **Variational autoencoders** draw a latent code and then simulate from a decoder distribution.
- For extremes, fidelity, diversity and novelty may remain essential, while **extrapolability**—i.e., tail-probability and quantile inference, and simulation beyond the well-observed range under explicit EVT-compatible assumptions—also has a role to play.

Generative Adversarial Networks

EV-GAN and Extreme-Event Simulation



Allouche, Girard and Gobet (2022)

- **GANs** learn a generator through an adversarial comparison between observed and simulated samples, without an explicit likelihood.
- **Tail limitation:** A finite ReLU generator is globally Lipschitz. With Gaussian latent inputs, each output marginal is light-tailed and cannot have an asymptotically regularly varying tail.
- **EV-GAN:** Allouche, Girard and Gobet introduce an EVT-informed generator for extreme-event simulation. This construction will be developed in tomorrow's lecture.

References

Foundations and Architectures

- Bishop, C. M. and Bishop, H. (2024). *Deep Learning: Foundations and Concepts*. Springer.
- Efron, B. and Hastie, T. (2016). *Computer Age Statistical Inference: Algorithms, Evidence, and Data Science*. Cambridge University Press.
- Papamakarios, G., Nalisnick, E., Rezende, D. J., Mohamed, S. and Lakshminarayanan, B. (2021). “Normalizing Flows for Probabilistic Modeling and Inference.” *Journal of Machine Learning Research*, 22(57), 1–64. arXiv:1912.02762.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. and Polosukhin, I. (2017). “Attention Is All You Need.” *Advances in Neural Information Processing Systems*, 30.
- Kingma, D. P. and Welling, M. (2014). “Auto-Encoding Variational Bayes.” *International Conference on Learning Representations*. arXiv:1312.6114.

References

Extremes and Applications

- Boehmke, B. and Greenwell, B. (2020). *Hands-On Machine Learning with R*, Chapter 19.
- Hickling, T. and Prangle, D. (2025). "Flexible Tails for Normalizing Flows." *Proceedings of the 42nd International Conference on Machine Learning*, PMLR 267.
- Lafon, N., Naveau, P. and Fablet, R. (2026). "A VAE Approach to Sample Multivariate Extremes." *Journal of Statistical Computation and Simulation*. arXiv:2306.10987.
- de Carvalho, M. and Richards, J. (2026). "Bridging Heavy Tails and Artificial Intelligence." *Extremes*, 29(2), 121–128.
- Wessel, J. B., Murphy-Barltrop, C. J. R. and Simpson, E. S. (2026). "Generative Machine Learning for Multivariate Angular Simulation." *Extremes*, 29(2), 195–243.
- IBM Research. "Generative Machine Learning for Extreme Weather Generation." <https://research.ibm.com/projects/generative-machine-learning-for-extreme-weather-generation>.